The `Lebowski` class defines a get property hook on the `public string $status` property. Whenever this property is accessed, the hook block is called – which must return data of the same type as the property. To clients of the class, this looks like seamless property access, and the hook will also execute in `$this` context, as shown by the call to `::printStatus()`. The set property hook shown here is technically redundant and is included simply to show the full syntax of the feature.

Constructors

It's quite common for at least some of the initial state of an object to be controlled by client code, there is a method available for passing data at instantiation time: the *constructor*.

A constructor is a special method that is executed every time a new instance is created and is commonly used to set the initial state of the object (so common, in fact, that PHP 8 introduced a feature to promote constructor parameters to object properties, we'll see this in a moment). Here's a new `QuoteBot` with a constructor, you'll notice that it looks very much like a setter method. For extra safety there's a quick check of the `$quoteStash` array size when the constructor executes, used to store a new class property that ensures we'll never go out of bounds when making use of the array:

```
class QuoteBot
{
    private array $quoteStash;
    private int $numQuotes;

    public function __construct(array $stash)
    {
        $this->quoteStash = $stash;
        $this->numQuotes = count($stash) - 1; // used as array index
    }

    public function getQuote(): string
    {
        return $this->quoteStash[rand(0, $this->numQuotes)];
    }
}
```